ObsidianDQ

An Agentic Data-Quality Investigation and Remediation System

PROJECT REPORT

Submitted by

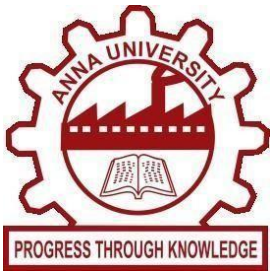
JEGADEESWARAN D **2116231801069**

KARTHICK S **2116231801079**

KOUSHAL V **2116231801084**

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS),
CHENNAI – 602 105

MAY-2026

BONAFIDE

This is to certify that the project entitled **"ObsidianDQ: Hybrid Agentic Data Quality Investigation and Remediation System"** is a bonafide record of independent project work carried out under academic and technical supervision. The software system, architecture, algorithms, and evaluation benchmarks described in this report were developed and verified to demonstrate an autonomous, auditable, and human-governed data observability and self-healing framework. To the best of our knowledge, the matter embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

Guide's Signature

Head of Department's Signature

ACKNOWLEDGEMENT

The successful development and empirical evaluation of **ObsidianDQ** would not have been possible without the valuable support, guidance, and resources provided by several individuals and open-source communities.

First and foremost, I express my sincere gratitude to my project supervisor for their invaluable guidance, technical insights, constructive feedback, and continuous encouragement throughout the conceptualization, architectural design, implementation, and evaluation of this project.

I would also like to extend my appreciation to the open-source communities and contributors behind **LangGraph, FastAPI, DuckDB, sqlglot, Next.js, and Tailwind CSS**. Their robust frameworks, libraries, and developer tools provided essential foundations for the development of the deterministic and agentic architecture implemented in ObsidianDQ.

Finally, I sincerely thank my peers, colleagues,faculty for their continuous support, valuable feedback, and patience throughout the research, development, testing, and benchmark evaluation phases of this project.

ABSTRACT

Modern enterprise data architectures rely on multi-stage Extraction, Load, and Transformation (ELT/ETL) pipelines feeding analytical dashboards, feature stores, and decision-support engines. However, silent data corruptions—such as missing primary keys, out-of-range timestamps, invalid categorical domains, negative numeric fields, and join-key mismatches—frequently propagate undetected through data lineage graphs, resulting in degraded analytical outputs and costly business remediation. Traditional data quality (DQ) systems rely on static assertion suites that detect failures but lack automated root-cause reasoning, contextual planning, and self-healing capabilities. Conversely, naive Large Language Model (LLM) applications deployed over data warehouses introduce risks of hallucinations, unpredictable write side-effects, syntax errors, and uncontrolled data loss.

This report presents **ObsidianDQ,** a local Hybrid Agentic Data Quality Investigation and Remediation System. ObsidianDQ couples deterministic execution engines with multi-turn, tool-using LLM agent graphs. The architecture separates decision proposal from execution authority: deterministic modules perform dataset profiling, rule-based anomaly detection, Directed Acyclic Graph (DAG) lineage traversal, Abstract Syntax Tree (AST) SQL self-healing, data quarantine, and post-remediation re-verification. Simultaneously, a stateful multi-agent LangGraph workflow comprising an **Investigation Agent**, **Root-Cause Agent**, **Planning Agent**, **Triage Agent**, and **Critic Agent** inspects evidence through read-only tools, formulates structured containment plans, and evaluates execution safety.

To eliminate rogue mutations, ObsidianDQ incorporates a stateful **Human-in-the-Loop (HITL)** governance checkpoint via LangGraph interrupts, triggering approval requirements when risk is classified as HIGH, proposal confidence drops below 70%, or the Critic demands revision. Quarantined records are isolated to distinct Parquet partitions, leaving raw source datasets immutable. Repaired transformation queries are validated via sqlglot AST dialect parsing. Persistence is managed via an in-process, thread-safe **DuckDB** database. Empirical evaluation across an automated 33-contract test suite and a controlled 20-incident benchmark establishes a **100.0% Controlled Incident Handling Success Rate** and **100.0% Controlled Verification Consistency**, while enforcing rigorous grounding rules that distinguish deterministic guarantees from live-LLM reasoning performance.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

Enterprise data operations have shifted from monolithic batch architectures to distributed, multi-hop ELT/ETL topologies. Raw event streams, vendor extracts, and transactional snapshots land in data lakehouses (in formats such as Parquet and CSV) and undergo multi-stage staging, transformation, and dimensional modeling via SQL views and orchestration engines.

In this interconnected environment, data quality is an operational prerequisite. Modern organizations depend on clean data for real-time executive dashboards, automated financial reporting, and machine learning feature pipelines. As data pipelines grow in depth and breadth, inter-table dependencies form intricate Directed Acyclic Graphs (DAGs). A schema variation, format corruption, or business-logic violation introduced at an upstream ingestion stage cascades downstream, multiplying its blast radius across derived tables and reporting layers.

1.2 Problem Statement

Despite the critical nature of data reliability, existing data quality management paradigms exhibit fundamental architectural vulnerabilities:

1. Passive, Alert-Only DQ Tooling: Established tools (e.g., Great Expectations, Soda, Monte Carlo) execute rigid assertion checks. When a test fails, they emit alerts via webhooks or Slack. They cannot autonomously investigate upstream lineage, query data slices dynamically, synthesize contextual root causes, or heal corrupted downstream SQL code.
2. Uncontrolled LLM Hallucination and Risk: Recent attempts to introduce Generative AI into data engineering typically employ unconstrained LLM agents equipped with database write privileges. This introduces acute operational risk: LLMs hallucinate non-existent table schemas, execute destructive row updates, drop columns, or propose unvalidated SQL syntax.
3. Lack of Truthful Verification and Closed-Loop Governance: Existing automated healing scripts rarely re-verify remediated artifacts against the original rule suite, frequently declaring success without confirming whether secondary errors were introduced. Furthermore, they lack checkpointed human-in-the-loop (HITL) approval gates capable of pausing execution state across process boundaries.

1.3 Objectives

To resolve the dichotomy between rigid deterministic tools and unsafe LLM agents, ObsidianDQ was developed with the following primary objectives:

- Architect a Hybrid Agentic Paradigm: Construct an execution graph where deterministic Python engines retain exclusive authority over data reading, AST parsing, and file writes, while LLM agents act strictly as read-only diagnostic investigators, planners, and safety critics.
- Implement Multi-Stage Lineage Root Cause Analysis (RCA): Map data dependencies via lineage graphs to isolate the earliest corruption origin and calculate downstream blast radius.
- Provide Safe, Non-Destructive SQL Healing: Build an AST-grounded SQL healing node using sqlglot to repair broken column identifiers (e.g., reconciling cust_id with customer_id) without altering analytical semantics.

- Enforce Human-in-the-Loop (HITL) Governance: Implement stateful graph interruptions that halt execution whenever remediation risk is HIGH, proposal confidence is < 0.7, or an issue is flagged for human intervention.
- Ensure Truthful Post-Remediation Verification: Quarantine anomalous records to dedicated Parquet partitions, generate a clean derived artifact, re-run all DQ detection checks, and fail truthfully if anomalies remain.
- Maintain Audit-Ready Persistent Observability: Store end-to-end execution telemetry, agent tool calls, incident memory, and evaluation records within an embedded DuckDB database.

2. RELATED WORK AND MOTIVATION

Comparative Landscape - Data reliability and automated remediation exist at the intersection of data observability, compiler techniques, and autonomous AI agents.

Capability Dimension	Traditional Assertion Suites (e.g., Great Expectations, Soda)	Naive LLM Text-to-SQL / Remediation Agents	ObsidianDQ Hybrid Agentic Architecture (This Work)
Anomaly Detection Method	Rigid assertion tests; manual config	Unbounded prompt-based file scanning	Vectorized, deterministic Python rule engine
Root-Cause Investigation	None (Static alerts sent to Slack/Email)	Hallucinatory text explanation without proof	Lineage DAG traversal + dynamic read-only tool calling
SQL Healing Mechanism	None (Manual query refactoring required)	Unconstrained LLM rewrite (Syntax errors frequent)	AST-grounded dialect parsing via sqlglot
Warehouse Write Permissions	Read-only detection	Direct read/write credentials (Critical Risk)	Zero direct source writes; isolated quarantine partitions
Human-in-the-Loop Governance	External ticketing systems (Jira)	Brittle prompt pauses or unassisted autonomy	Stateful graph checkpoint interrupts (MemorySaver)
Verification Cycle	Manual pipeline re-run	Assumed success upon generation	Automated post-remediation verification on cleaned file
Adversarial Safety Audit	None	None	Dedicated Critic Agent with bounded retry (≤ 1)
Failure Recovery Mode	Hard pipeline failure	Unpredictable loop or silent corruption	Graceful fail-closed deterministic fallback

Motivation for the Hybrid Paradigm - Purely deterministic pipelines guarantee safety and speed but lack adaptive intelligence when diagnosing multi-table anomalies. Conversely, purely autonomous LLM agents provide high reasoning flexibility but lack determinism, idempotence, and formal verification. ObsidianDQ bridges this gap by enforcing architectural containment: the LLM is given "eyes but no hands." The agent may call read-only diagnostic tools (such as sampling rows, checking column distributions, traversing lineage nodes, and querying historical incident memories) to propose actions. However, physical transformations—such as quarantining rows into data/quarantine/ and normalizing SQL ASTs—are executed exclusively by vetted deterministic Python algorithms.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

ObsidianDQ is designed as a governed hybrid architecture that combines deterministic data-quality processing with LLM-assisted investigation and decision support. The architecture separates **reasoning and recommendation from execution authority**. Deterministic components are responsible for data profiling, rule-based quality detection, lineage traversal, SQL parsing and validation, supported remediation, quarantine generation, post-remediation verification, and persistence. LLM-based agents operate within controlled boundaries and are primarily responsible for investigating evidence, reasoning about possible root causes, preparing remediation plans, proposing triage actions, and reviewing those proposals.

The system is organized into three major application layers:

1. **Core Agentic Engine (src/agent/)** The core engine implements the LangGraph workflow and maintains the shared AgentState. It coordinates deterministic processing, agent-based investigation, human approval, remediation, and verification. The graph controls the order in which each stage is executed and determines whether the workflow proceeds automatically, requires revision, or pauses for human approval.
2. **Backend Service (backend/main.py)** The FastAPI backend provides the application interface between the frontend and the execution engine. It handles dataset and configuration uploads, pipeline execution, approval or rejection of human-review requests, quarantine actions, health information, and serialization of pipeline telemetry.
3. **Observability Dashboard (frontend/)** The frontend is implemented using Next.js 14, React, Tailwind CSS, Framer Motion, and @xyflow/react. It provides the interface for configuring a run, monitoring pipeline execution, inspecting detected issues, viewing lineage, reviewing agent recommendations, approving or rejecting high-risk actions, and examining SQL transformation differences and execution diagnostics.

System Architecture

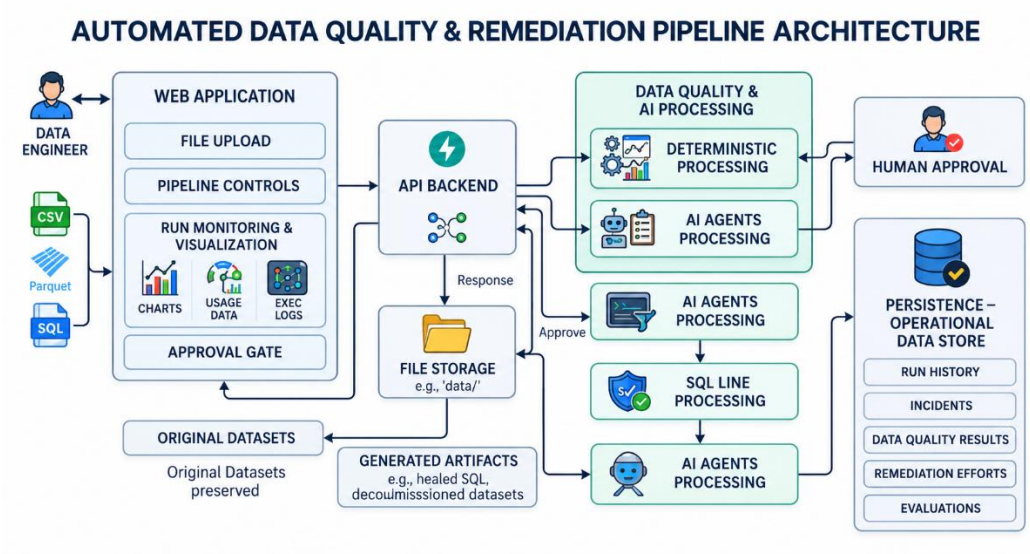


Figure 3.1: Overall ObsidianDQ System Architecture

The system architecture figure should show the following major boundaries: The important architectural boundary is that **LLM agents do not directly perform destructive dataset modifications**. Agent outputs are converted into structured proposals and are subsequently evaluated by workflow rules, human approval gates where required, and deterministic execution components.

3.2 Pipeline Workflow

The pipeline is implemented as a stateful LangGraph execution graph governed by AgentState. The workflow combines deterministic processing with specialized agent stages. Each stage receives the current state, performs its defined operation, and returns structured information for subsequent stages.

The workflow consists of the following stages.

1. stage_profile

The profiling stage loads the target CSV or Parquet dataset and extracts structural information such as the available columns, data types, row counts, and other basic profiling statistics required by subsequent DQ checks.

The default demonstration dataset is data/raw/stg_orders.parquet.

2. dq_detect

The DQ detection stage performs deterministic validation using the implemented data-quality rules. It evaluates the staging dataset and, where available, performs an upstream evaluation pass over the configured upstream dataset.

The detector produces structured issue records containing information such as the rule identifier, affected column or scope, severity, and detected record count.

The implemented rules include:

- NOT_NULL
- NO_DUPLICATES
- PRICE_NON_NEGATIVE
- VALID_STATUS
- UNIQUE_ORDER_ID
- ORDER_DATE_RANGE

This stage does not depend on LLM reasoning.

3. lineage_rca

The lineage stage reads the configured lineage graph from data/lineage/lineage.json. It constructs upstream and downstream relationships and traverses the graph to identify relevant ancestors, descendants, and downstream impact.

The resulting lineage information is supplied to subsequent investigation and root-cause stages.

The lineage stage identifies **candidate upstream origins and downstream impact**; it does not by itself prove the semantic root cause of a data-quality incident.

4. investigation_agent

The Investigation Agent performs evidence-oriented analysis using the configured LLM provider and read-only diagnostic tools.

The agent can inspect:

- Sample offending records
- Column statistics
- Lineage context
- Similar historical incidents

The purpose of this stage is to gather and interpret evidence rather than modify data.
When the LLM is unavailable or a call fails, the system uses the defined deterministic fallback behavior instead of allowing the failed LLM call to directly affect the dataset.

5. root_cause_agent

The Root-Cause Agent receives the detected issues, lineage information, and investigation evidence. It produces a structured assessment of the likely upstream cause and supporting reasoning.
The result is treated as an **agent-generated hypothesis supported by available evidence**, rather than as independently verified ground truth.
When the LLM is unavailable, the system can use the configured lineage-based heuristic and explicitly marks upstream causality as unproven.

6. planning_agent

The Planning Agent converts the investigation findings into a structured remediation plan.
The canonical plan contains four stages:
INVESTIGATE → SQL_HEAL → QUARANTINE → VERIFY
The planning stage also assigns an operational risk classification of:

- LOW
- MEDIUM
- HIGH

The plan is a proposal for subsequent workflow execution and is not itself a data modification operation.

7. triage_agent

The Triage Agent maps detected issues to a defined set of actions:

- AUTO_QUARANTINE
- FLAG_FOR_REVIEW
- IGNORE_TRANSIENT
- ESCALATE_UPSTREAM

Each proposal is accompanied by a confidence value in the range [0.0, 1.0].
When the LLM cannot successfully produce a valid triage result, unresolved issues are assigned FLAG_FOR_REVIEW with confidence 0.0, causing the human-approval condition to be evaluated.

8. critic_agent

The Critic Agent reviews the proposed root cause, triage actions, and remediation plan for logical inconsistencies or safety concerns.
The critic produces one of two outcomes:

- APPROVED

- REVISION_REQUIRED

When revision is required, the workflow returns to the Investigation Agent for a bounded retry. The retry count is limited to prevent an uncontrolled agent loop. If the revision process is exhausted, the workflow proceeds toward human review.

If the Critic LLM call is unavailable, the current implementation uses an approval fallback while recording the corresponding warning. This is a known limitation of the current prototype and is not treated as a fail-closed safety guarantee.

9. human_review_queue

The human-review stage acts as the governance checkpoint for actions that require explicit human approval.

The workflow requires human review when the configured risk or proposal conditions indicate that automatic execution should not proceed. The principal conditions are:

$$RequireApproval = (Risk = HIGH) \vee (\exists p \in Proposals: p.action = FLAG_{FOR_{REVIEW}}) \vee p.confidence < 0.70) \vee (Critic = REVISION\_EXHAUSTED$$

When the condition is satisfied, the LangGraph workflow pauses at the human-review checkpoint. The current implementation uses LangGraph MemorySaver together with the application run state to retain the paused execution state in memory.

The frontend presents the relevant incident information, proposed actions, evidence, and impact information to the data engineer.

10. sql_healer

The SQL-healing stage performs constrained SQL transformation using sqlglot.

The transformation query is parsed into an Abstract Syntax Tree using the DuckDB SQL dialect. The implementation validates the query structure and performs the supported identifier correction in the demonstration workflow.

For example, the demonstration transformation contains an invalid reference to: c.cust_id

which is corrected to: c.customer_id

The resulting SQL is validated and written as a separate healed query rather than overwriting the original transformation query.

This component should therefore be understood as a **constrained AST-based SQL repair mechanism**, not as a general-purpose semantic SQL repair system.

11. remediation

The remediation stage applies the approved and supported containment actions without modifying the original source dataset in place.

Where quarantine is required, affected records are separated into a quarantine artifact. Clean records are written to a derived cleaned dataset.

The demonstration paths are: data/quarantine/stg_orders_quarantine.parquet and data/cleaned/stg_orders_cleaned.parquet

The exact remediation action depends on the detected rule and the triage decision. Rules that do not have an implemented automatic repair operation are detected and reported rather than being represented as automatically repaired.

12. verify_agent

The verification stage independently re-runs the deterministic DQ detection process against the generated cleaned artifact.

Verification succeeds only when the remaining issue count satisfies the defined verification criterion. If unresolved issues remain, the system does not silently report the artifact as clean.

13. verification_recovery

If verification fails, the recovery stage records the failure condition as:

VERIFICATION_FAILED

and records the requirement for further recovery or investigation.

The current implementation reports the failure but does not automatically roll back or delete the generated artifacts.

14. guardrails

The final guardrail stage checks the consistency of the workflow state and records the relevant run, incident, evidence, remediation, and evaluation information in DuckDB.

The guardrail stage therefore provides the final control and persistence boundary for the completed workflow.

3.3 LLM Providers and Fallback Strategy

ObsidianDQ uses a provider-neutral LLM interface implemented in src/agent/utils/llm.py. The provider layer allows the agent workflow to operate with an available LLM provider while retaining deterministic fallback behavior when the provider cannot be used.

Provider Selection

The application checks for the configured providers in the following order:

1. **Groq** — selected when GROQ_API_KEY is available. The configured default model is openai/gpt-oss-20b.
2. **Gemini** — selected when GEMINI_API_KEY or GOOGLE_API_KEY is available. The configured model is gemini-2.5-flash.
3. **Offline Fallback** — selected when no supported LLM credentials are available.

The provider-selection mechanism is intended to make the agent layer replaceable without changing the overall workflow architecture.

Generation Configuration

The configured LLM temperature is: temperature = 0.1

The low temperature is intended to reduce unnecessary output variation for structured agent tasks. It does **not** guarantee deterministic LLM responses.

Rate-Limit Handling

Groq requests include retry handling for HTTP 429 rate-limit responses. The implementation uses bounded retry attempts with increasing delays before treating the LLM invocation as unsuccessful.
This mechanism improves resilience to temporary rate limiting but does not guarantee successful external API execution.

Deterministic Fallback Behavior

When an LLM call fails, times out, becomes unavailable, or exceeds the available quota, the system uses stage-specific fallback behavior.

- **Investigation Agent:** falls back to deterministic evidence extraction.
- **Root-Cause Agent:** uses the available lineage-based heuristic and records that upstream causality has not been proven.
- **Triage Agent:** unresolved issues are assigned FLAG_FOR_REVIEW with confidence 0.0.
- **Critic Agent:** the current implementation uses an approval fallback when the critic invocation is unavailable and records a warning.

The final behavior therefore depends on the combination of the LLM result, deterministic fallback logic, risk conditions, and human-review requirements.
The fallback mechanism is intended to maintain workflow continuity and truthful reporting; it should not be interpreted as equivalent to successful LLM reasoning.

3.4 Human-in-the-Loop Design

ObsidianDQ incorporates a Human-in-the-Loop (HITL) checkpoint to provide human oversight for remediation actions that require explicit review. The approval conditions and routing logic are evaluated by the workflow before the execution stage, as described in the pipeline workflow.
When human review is required, the LangGraph workflow transitions to the human_review_queue checkpoint and pauses execution. The current implementation uses LangGraph MemorySaver together with the application's in-process run-state storage to retain the paused workflow state using the associated run_id.

Human Review Process

The web dashboard presents the information required by the data engineer to evaluate the proposed action. The review interface provides:

- Detected data-quality issues and their severity
- Investigation evidence
- Root-cause assessment
- Proposed triage action and confidence
- Lineage information and downstream impact
- Proposed remediation plan

The reviewer can then either approve or reject the pending execution.

Approval and Rejection Flow

The LLM-generated recommendation is treated as a proposal rather than a direct execution command. When human approval is required, the workflow pauses at the Human Approval stage. The reviewer then determines whether the proposed action should proceed.

If approved, the paused workflow resumes and proceeds through SQL healing or deterministic remediation, followed by quarantine or clean artifact generation, post-remediation verification, guardrails validation, and persistence. The original source dataset is not directly modified by the agent-generated recommendation.

If rejected, the workflow records the APPROVAL_REJECTED status and terminates. The pending remediation operation is not executed, and the original source dataset remains unchanged.

This flow establishes the separation between LLM-based decision support and deterministic execution, with human approval controlling whether the proposed remediation is allowed to proceed.

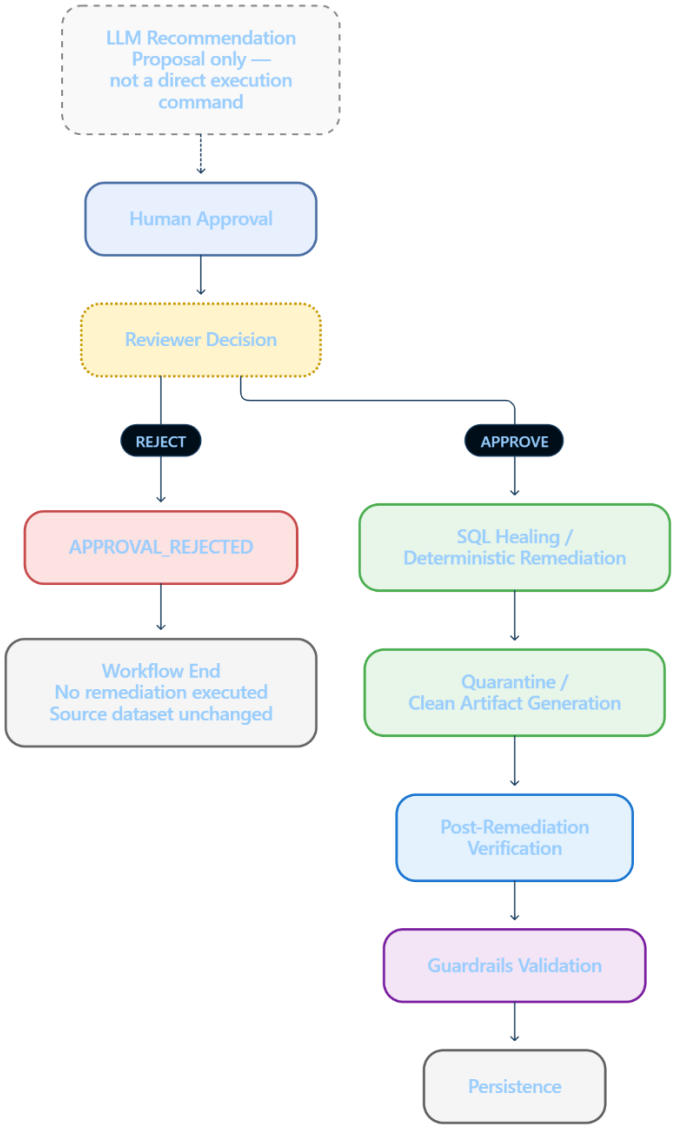


Figure 3.4: Approval and Rejection Flow

HITL Limitation

The current HITL implementation is process-local because workflow checkpoints and run-state information are maintained in memory. Consequently, a paused approval request is not durable across server restarts.

For production-scale or distributed deployment, the checkpointing mechanism would need to be replaced or extended with a persistent state store, such as PostgreSQL, to maintain approval state across application restarts and multiple backend instances.

4. Methodology

4.1 Data-Quality Detection Rules

Data-quality detection (src/agent/nodes/dq_detect.py) is implemented using vectorized Pandas operations over in-memory dataset buffers, guaranteeing sub-second evaluation. Six core expectation rules are enforced:

1. NOT_NULL

- **Column/Scope:** Any dataset column (customer_id, price, order_date)
- **Severity:** MEDIUM
- **Expectation:** Detect NULL or missing values.
- **Containment:** Flag for human review; quarantine if the column has high downstream impact.
- **Impact:** Prevents incomplete records from downstream processing.

2. NO_DUPLICATES

- **Column/Scope:** Full row tuple
- **Severity:** LOW
- **Expectation:** Detect repeated rows based on complete row equality.
- **Containment:** Flag for monitoring or human review; no automatic deduplication.
- **Impact:** Identifies identical row retransmissions.

3. PRICE_NON_NEGATIVE

- **Column/Scope:** price
- **Severity:** HIGH
- **Expectation:** Require price ≥ 0.
- **Containment:** Quarantine negative prices when permitted; otherwise flag for review.
- **Impact:** Prevents invalid price records from downstream calculations.

4. VALID_STATUS

- **Column/Scope:** status
- **Severity:** MEDIUM
- **Expectation:** Status must belong to the configured valid set.
- **Containment:** Flag for review and apply the configured action.
- **Impact:** Prevents invalid categorical values from downstream business logic.

5. UNIQUE_ORDER_ID

- **Column/Scope:** order_id
- **Severity:** HIGH
- **Expectation:** Require Distinct(order_id) = N.
- **Containment:** Quarantine or flag duplicate order_id records.
- **Impact:** Prevents conflicting order records from downstream joins and aggregations.

6. ORDER_DATE_RANGE

- **Column/Scope:** order_date
- **Severity:** HIGH
- **Expectation:** Require order_date ≥ 2020-01-01 and the configured upper boundary, if defined.

- **Containment:** Flag out-of-range dates; quarantine only where supported.
- **Impact:** Detects invalid/corrupted dates, including 1970-01-01.

In addition to evaluating the primary staging table, dq_detect performs an upstream evaluation pass over data/raw/raw_customers.csv if present, merging upstream anomalies into the pipeline state with stage attribution tags.

4.2 Lineage-Based Root Cause Analysis

Modern pipeline failures often manifest downstream from the root cause. ObsidianDQ parses Directed Acyclic Graphs defined in lineage.json into forward and reverse adjacency sets:

$$G = (V, E),\ \text{Upstream}(v) = \{u \in V \mid (u, v) \in E\},\ \text{Downstream}(v) = \{w \in V \mid (v, w) \in E\}$$

1. Ancestry & Descendant Traversal: Breadth-first traversal identifies all transitive upstream ancestors $\text{Ancestors}(v)$ and downstream descendants $\text{Descendants}(v)$.
2. Blast Radius Computation: The blast radius is quantified as $|\ \text{Descendants}(v)\ |$, denoting the exact number of downstream data products, views, and dashboards jeopardized by corruption in v.
3. Earliest Origin Isolation: Candidate root causes are ranked by upstream depth. Nodes with zero upstream parents ($\text{InDegree}(u) = 0$) within the ancestor subgraph are isolated as primary root-cause candidates.

4.3 Agentic Investigation, Planning and Triage

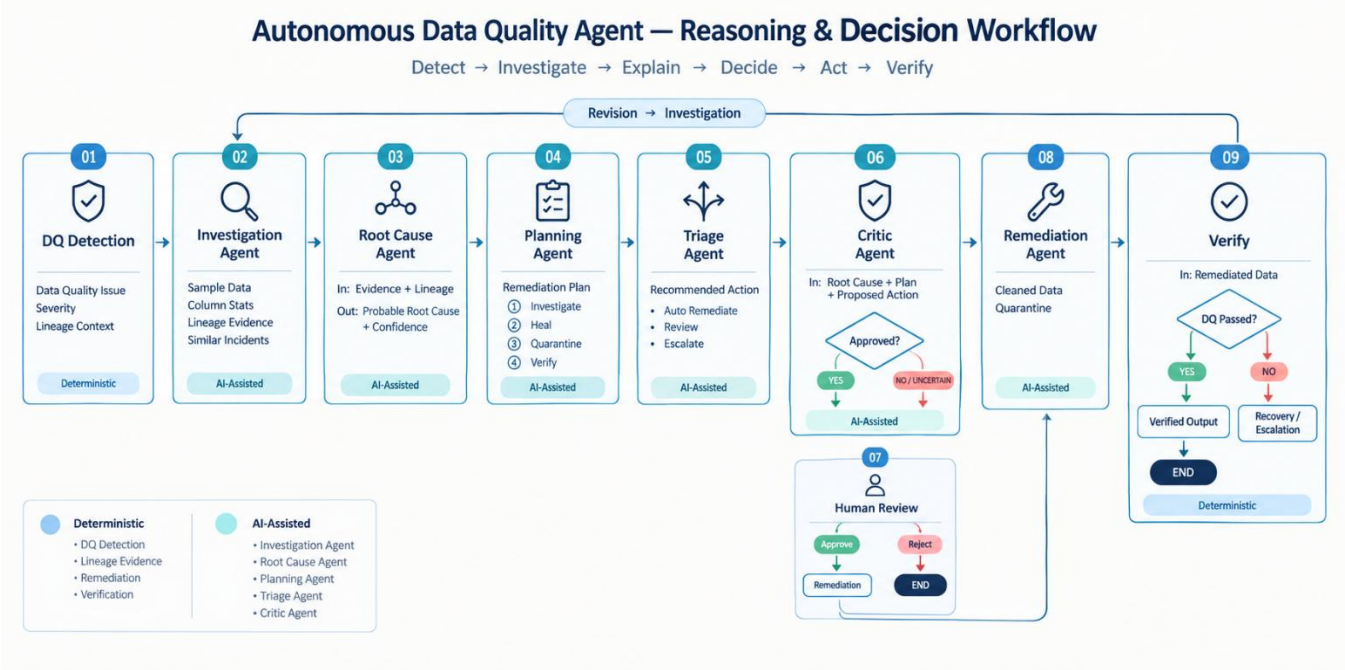


Figure 4.3: Agentic Investigation, Planning and Triage

The multi-agent subsystem operates sequentially through specialized roles:

- **Investigation Agent:** Operates over multiple conversational turns using Groq function-calling. It dynamically chooses from four read-only diagnostic tools: get_sample_rows (extracting up to 5 matching corrupted tuples), get_column_stats (computing null counts, distinct values, and data

types), get_lineage_context (retrieving topological lineage),
and search_similar_incidents (querying past incident resolutions from DuckDB).

- **Root-Cause Agent:** Takes the investigation trail, critic feedback, and lineage graph to determine whether upstream corruption caused the staging anomaly, outputting a structured conclusion with a confidence score.
- **Planning Agent:** Enforces that every remediation attempt contains the mandatory four step types (INVESTIGATE, SQL_HEAL, QUARANTINE, VERIFY), determining overall operational risk based on issue severity.
- **Triage Agent:** Generates structured action proposals for each detected issue ID (RULE:column), assigning actions from the closed set: Action $\in$ {AUTO_QUARANTINE, FLAG_FOR_REVIEW, IGNORE_TRANSIENT, ESCALATE_UPSTREAM}
- **Critic Agent:** Audits triage recommendations against issue severity and lineage logic. If the critic identifies logical discrepancies, it outputs REVISION_REQUIRED. A graph edge routes execution back to the Investigation Agent. To prevent infinite loops, critic_retry_count is bounded: after one failed revision attempt, the system automatically forces a human review interrupt.

4.4 SQL Healing, Remediation, and Verification

ObsidianDQ uses sqlglot to parse SQL into a DuckDB AST instead of relying on unrestricted LLM-generated SQL.

- **Structural validation:** Detects syntax and structural SQL issues.
- **Constrained repair:** Corrects supported schema-reference errors, such as c.cust_id → c.customer_id, and validates the resulting SQL.

Data Quarantine & Remediation

- Source files are treated as **immutable** and are never overwritten.
- Approved quarantine actions isolate affected rows into a quarantine Parquet file.
- Valid rows are exported to a separate cleaned Parquet file.

Post-Remediation Verification

The verification stage re-runs the deterministic DQ rules on the cleaned artifact.

- **0 remaining issues:** verification_passed = True
- **Issues remain:** verification_passed = False → VERIFICATION_FAILED → recovery handling

5. IMPLEMENTATION

5.1 Backend

The backend application is implemented in **Python 3.10+**, using **FastAPI** for the REST API layer and **LangGraph** for managing the multi-agent workflow.

State Management

The workflow state is defined in src/agent/state.py through AgentState, which uses typing_extensions.TypedDict.

AgentState contains **48 strongly typed state variables** that maintain information throughout the pipeline execution. These variables cover:

- Pipeline paths and configuration
- Data profiling metrics
- Detected data-quality issues
- Data-lineage maps
- Agent execution traces
- SQL repair differences
- Remediation paths
- Verification results

This shared state allows the different stages of the LangGraph workflow to exchange structured information while maintaining a consistent execution context.

RESTful API Endpoints

The backend exposes the following REST endpoints:

GET /api/health

Reports the current backend status, including:

- Server status
- Active LLM provider (groq, gemini, or None)
- LLM availability

POST /api/pipeline/upload

Handles multipart file uploads.

The endpoint accepts:

- CSV files
- Parquet files
- SQL files
- Lineage JSON files

Uploaded files are stored in: data/uploads/

The endpoint also performs validation and provides previews of the uploaded content.

POST /api/pipeline/run
Receives the pipeline configuration, starts the pipeline through run_pipeline(), and returns the serialized **3-tier telemetry payload** containing the pipeline execution information.

POST /api/pipeline/approve
Handles human approval decisions for interrupted workflow executions.
It accepts: {run_id, issue_id, decision, action}
The endpoint updates the corresponding checkpointer state in MemorySaver and resumes the interrupted LangGraph thread.

POST /api/pipeline/quarantine
Provides a standalone **1-click quarantine execution endpoint** for isolating affected data.

Persistent Storage
Persistent database functionality is implemented in: src/agent/utils/db.py
The previous JSONL-based storage mechanism was replaced with an embedded **DuckDB** database: data/obsidiandq.duckdb
The database layer maintains a **thread-safe, persistent singleton connection** and includes retry handling to reduce lock-contention problems when multiple API worker threads access the database.
The database contains six structured tables:

1. runs — Stores pipeline execution information.
2. incidents — Stores detected data-quality incidents.
3. dq_results — Stores data-quality detection results.
4. rca_evidence — Stores evidence collected during root-cause analysis.
5. remediation_results — Stores remediation outcomes.
6. evaluation_records — Stores evaluation and scoring information.

5.2 Frontend
The frontend is implemented using **Next.js 14 with the App Router**, **React**, **Tailwind CSS**, and **Framer Motion** and the interface follows a guided **three-step workflow** that takes the user from source configuration through execution and monitoring.
Step 1: Source Configuration
Implemented in:
UploadStep.tsx
This step provides drag-and-drop upload areas for:

- Dataset files
- SQL queries
- Lineage graphs

It also provides a **1-click "Demo Preset"** option for loading the predefined demonstration configuration.
Step 2: Confirmation Ticket
Implemented in:

ConfirmationTicketStep.tsx
This step provides a pre-execution flight check before the pipeline starts.
It displays information such as:

- File hashes
- Target stages
- Schema parameters

The purpose of this step is to allow the user to verify the configured inputs before execution.

Step 3: Run Console

Implemented in:
RunConsole.tsx
The Run Console provides a unified monitoring environment containing **six tabs**.

1. Overview

Provides a high-level view of the pipeline execution through:

- Health Score
- Scanned Records
- Issue Count
- Blast Radius

It also displays an executive narrative summary and provides quick-action buttons.

2. Issues

Provides an interactive DataTable containing:

- Issue types
- Affected columns
- Record counts
- Severity badges

An IssueDetailDrawer side panel provides additional information for the selected issue.

3. Lineage

Displays the pipeline's dependency graph using @xyflow/react.
The graph represents the health status of individual lineage nodes using:

- HEALTHY
- ATTENTION
- FAILED
- DEGRADED

4. Data

Provides a dataset preview showing the raw records together with profiling metrics such as:

- Null percentages
- Distinct counts
- Data types

This allows the user to examine the underlying data alongside the profiling results.

5. Actions

Displays the remediation and decision-making components, including:

- Multi-agent recommendation card
- Stateful ApprovalGate
- Animated RunTimeline

The ApprovalGate provides the human approval point within the workflow.

6. Details

Provides technical execution information, including:

- Pipeline execution diagnostics
- AST SQL diff comparison viewer
- Complete JSON technical trace

This tab exposes the detailed technical information behind the pipeline execution.

5.3 Testing Infrastructure

ObsidianDQ uses a **Pytest contract test suite** located in: tests/

The tests cover the core data-quality pipeline, agent workflow, API contracts, database operations, and evaluation logic.

test_agent.py:

Validates the core pipeline functionality, including:

- Input loading
- Rule detection accuracy
- Lineage graph traversal
- AST-based SQL healing
- Quarantine writing
- Guardrail validation

test_agentic_workflow.py:

Validates the LangGraph-based agent workflow, including:

- Workflow routing logic
- Fail-closed behavior when LLM API keys are unavailable
- Bounded critic retries
- Thread interruption and resumption cycles

test_agentic_core.py:

Tests the core agentic execution contract, including:

- The 4-step canonical planning contract
- Post-remediation verification on cleaned files
- Multi-turn tool-use mocking

test_api_contract.py:

Uses FastAPI's TestClient to verify that API responses conform to the defined **3-tier telemetry contract** both before and after human approval.

test_db.py:
Validates the DuckDB persistence layer, including:

- Database schema initialization
- Transaction handling
- Incident persistence
- Historical run retrieval

test_evaluation_logic.py: Validates the evaluation logic and ensures that **fallback executions cannot be incorrectly counted as successful live-LLM executions**.
This prevents fallback behavior from being reported as genuine LLM-based success during evaluation.

6. EVALUATION

6.1 Evaluation Approach

ObsidianDQ uses a controlled evaluation approach to distinguish **software and workflow correctness** from **LLM reasoning performance**. This distinction is important because a successful pipeline execution may result from deterministic fallback logic when a live LLM call is unavailable or unsuccessful.

The evaluation is implemented through the reproducible benchmark harness: evaluation/run_controlled_benchmark.py

The evaluation consists of two complementary levels.

1. System Contract Validation

The project uses Pytest to validate the implemented software and workflow contracts. The current repository contains **49 test functions** across the test suite.

These tests validate areas such as:

- Data and input loading
- Deterministic data-quality detection
- Lineage graph traversal
- LangGraph workflow routing
- Agent-state handling
- Structured planning
- Human-review interruption and resumption
- AST-based SQL healing
- Remediation and quarantine handling
- DuckDB persistence
- Post-remediation verification
- API response contracts
- Evaluation logic and fallback handling

The test suite is used to establish that the implemented components behave according to their defined contracts. A successful test therefore demonstrates **implementation and workflow correctness**, not LLM reasoning quality.

2. Controlled Benchmark Execution

A controlled benchmark processes **20 synthetic data-quality incidents**, identified as C01 through C20, through the compiled ObsidianDQ workflow.

The benchmark contains individual anomalies, compound anomalies, clean-data controls, lineage-related scenarios, human-approval cases, low-confidence cases, and verification-failure conditions.

The benchmark cases include:

Cases	Scenario
C01	Negative numeric price values (PRICE_NON_NEGATIVE)
C02	Invalid categorical order status (VALID_STATUS)
C03–C04	Missing foreign keys and null timestamps (NOT_NULL)

C05	Corrupted historical dates (ORDER_DATE_RANGE)
C06–C07	Complete duplicate rows and duplicate primary keys (UNIQUE_ORDER_ID)
C08–C10, C15	Compound mutations involving nulls, negative prices, and invalid dates
C11	Clean control dataset used to check for false positives
C16–C20	Upstream lineage propagation, high-risk human-approval pauses, low-confidence triage, and deliberate verification failures

The benchmark is executed using the configured pipeline modes, including **offline fallback** and **live-LLM configurations**. The resulting evaluation records are stored under: evaluation/results/

The purpose of this benchmark is not to claim general-purpose autonomous AI accuracy. Instead, it evaluates whether the implemented workflow reaches the **expected controlled outcome** for each predefined scenario.

6.2 Grounding Rules Applied During Evaluation

To prevent deterministic fallback behavior from being incorrectly reported as successful LLM reasoning, the evaluation follows six grounding rules.

1. Automated Test Pass Rate Measures Contract Coverage, Not LLM Quality

A successful Pytest result demonstrates that the implemented components satisfy their defined software and workflow contracts.

For example, the tests can establish that:

- graph routing works correctly,
- state transitions follow the expected structure,
- data transformations execute correctly,
- persistence operations work,
- API responses follow the defined contract.

However, these results do **not** measure the quality or accuracy of LLM reasoning. Therefore, the automated test pass rate is treated as a measure of **contract coverage and implementation correctness**, rather than an LLM performance metric.

2. Offline Handling Success Is Not Agent Task Success

When the system operates in offline_fallback mode, deterministic logic can continue handling supported data-quality issues even when a live LLM is unavailable.

Successful detection, containment, quarantine, or other deterministic handling in this mode demonstrates that the underlying workflow can operate under the defined fallback conditions.

It does not demonstrate successful LLM reasoning. Therefore, **Agent Task Success Rate is reported as NOT MEASURABLE for offline fallback executions**.

3. Verification Failure Is Correct Handling, Not Remediation Success

A remediation attempt does not necessarily mean that every anomaly can be automatically corrected.

For example, an out-of-range or corrupted date may be detected successfully while the current remediation logic does not have sufficient information to reconstruct the correct date.

In such cases, the verification stage re-runs the deterministic DQ checks against the resulting artifact. If the anomaly remains unresolved, the system records: VERIFICATION_FAILED
This is considered **correct verification behavior**, because the system has identified that the resulting artifact still violates the required data-quality condition and it is therefore not counted as successful remediation.

4. Live Metrics Require Successful Non-Fallback LLM Execution
LLM-specific metrics are calculated only when a live LLM client successfully executes without falling back to deterministic behavior and a configured API key alone is therefore not sufficient to classify an execution as a successful live-LLM run.
For example, when a configured LLM request fails because of a rate limit such as HTTP 429, the execution is recorded as: LLM_CONFIGURED_BUT_CALL_FAILED
This prevents failed live calls followed by deterministic fallback behavior from being incorrectly counted as successful LLM executions.

5. RCA Accuracy Is Not Measurable Without Semantic Ground Truth
Root-cause analysis requires a reliable reference answer against which the agent's explanation can be compared.
The current benchmark does not contain an independent, human-annotated semantic ground-truth dataset for root causes.
Therefore, metrics such as:

- RCA Precision
- RCA Recall
- RCA F1 Score

are reported as **NOT MEASURABLE**.
These metrics should only be introduced after an independently annotated RCA benchmark is available.

6. Failure Recovery Rate Is Not Measurable Without Fault Injection
Failure recovery cannot be meaningfully measured only by running normal benchmark cases.
A proper recovery evaluation requires controlled failures such as:

- process crashes,
- network timeouts,
- interrupted execution,
- or other injected runtime faults.

Because randomized fault injection is not part of the current benchmark, **Failure Recovery Rate is reported as NOT MEASURABLE**.
This prevents normal workflow execution from being incorrectly interpreted as evidence of fault-recovery capability.

7. RESULTS

7.1 Controlled Benchmark Findings

ObsidianDQ was evaluated using a controlled benchmark containing **20 synthetic data-quality incidents**. The benchmark covers individual and compound anomalies, clean-data controls, upstream lineage propagation, human-approval scenarios, low-confidence triage, and verification-failure conditions.

The evaluation results were recorded in: evaluation/results/

The evaluation focused on three primary aspects:

1. **Contract adherence**
2. **Controlled incident handling**
3. **Verification consistency**

Contract Adherence

The automated test suite validates the implemented workflow contracts across the major components of ObsidianDQ. The tests cover data loading, deterministic DQ detection, lineage traversal, agent-state routing, structured planning, human-review interruption and resumption, remediation handling, database persistence, API contracts, and post-remediation verification.

These tests establish that the implemented software behaves according to its defined contracts. They are therefore treated as evidence of **software and workflow correctness**, rather than evidence of LLM reasoning quality.

The current repository contains **49 test functions**. Any statement referring to a specific historical test-pass count should be tied to the corresponding dated test execution rather than presented as the current repository count.

Controlled Incident Handling

The 20 benchmark incidents were processed against the defined data-quality rules and workflow conditions.

The benchmark achieved a:

Controlled Incident Handling Success Rate: 100.0% (20/20)

This means that all 20 predefined cases reached their expected controlled outcome according to the benchmark criteria.

The expected outcome could include:

- Detection of the relevant issue
- Appropriate workflow handling
- Containment or quarantine were supported
- Human escalation where required
- Correct handling of low-confidence conditions
- Explicit reporting of verification failure where remediation was incomplete

The **100.0% result should not be interpreted as 100% LLM reasoning accuracy**. Some executions can use deterministic fallback behavior when live LLM calls are unavailable or unsuccessful.

Therefore, this result demonstrates the reliability of the implemented **controlled workflow and deterministic handling mechanisms within the benchmark scope**, rather than general-purpose autonomous reasoning capability.

Verification Consistency

After remediation, the verification stage re-runs the deterministic DQ checks against the generated cleaned artifact.

The benchmark achieved: **Verification Consistency: 100.0%**

This indicates that the verification mechanism correctly identified whether the resulting artifact satisfied the defined data-quality conditions or whether unresolved anomalies remained.

Importantly, verification does not assume that every remediation attempt must succeed.

When an anomaly cannot be completely resolved by the implemented remediation logic, the system reports: VERIFICATION_FAILED instead of incorrectly marking the artifact as valid.

Therefore, the 100% verification-consistency result represents **correctness of the verification decision**, not a claim that all 20 incidents were completely remediated.

Overall Evaluation Result

The evaluation demonstrates that ObsidianDQ can maintain deterministic control over the tested data-quality scenarios while using LLM-based components for investigation, root-cause reasoning, planning, triage, and critique.

The results provide evidence for:

- Workflow reliability
- Deterministic DQ handling
- Controlled remediation behavior
- Human-approval workflow handling
- Post-remediation verification consistency

However, these results should remain within the scope of the controlled benchmark. They do **not** establish general-purpose LLM reasoning accuracy, production-scale autonomous remediation, or fault-recovery performance.

In the latest live-Groq benchmark referenced in the evaluation records, the system recorded **0.0% Agent Task Success Rate**, **50.0% Tool Success Rate**, and multiple failed LLM calls that resulted in fallback behavior. These results reinforce the importance of separating deterministic workflow success from genuine live-LLM performance. Accordingly, the evaluation supports the claim that ObsidianDQ provides a **controlled hybrid data-quality investigation workflow**, rather than claiming unrestricted autonomous data-quality remediation.

7.2 Output

The following ASCII wireframes illustrate the core dashboard interfaces implemented in the Next.js frontend:

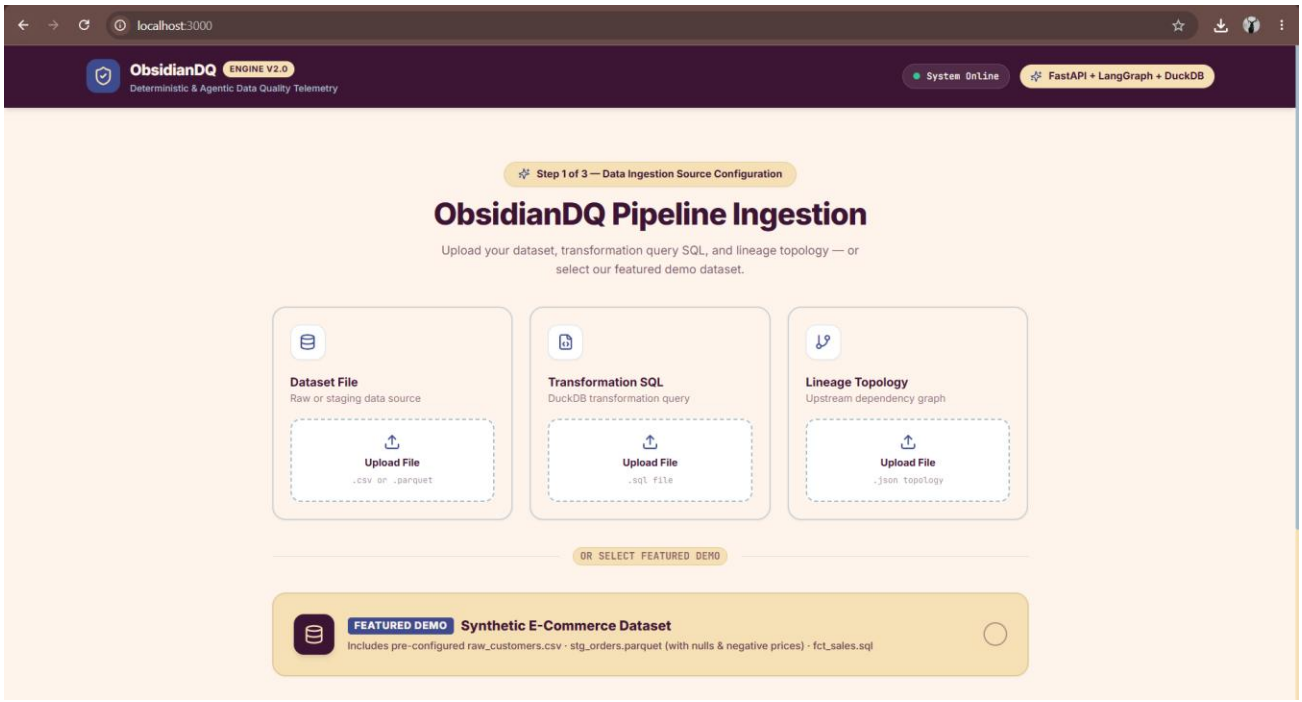


Figure 7.2: Screen 1Dataset Upload and Preset Selection (UploadStep.tsx)

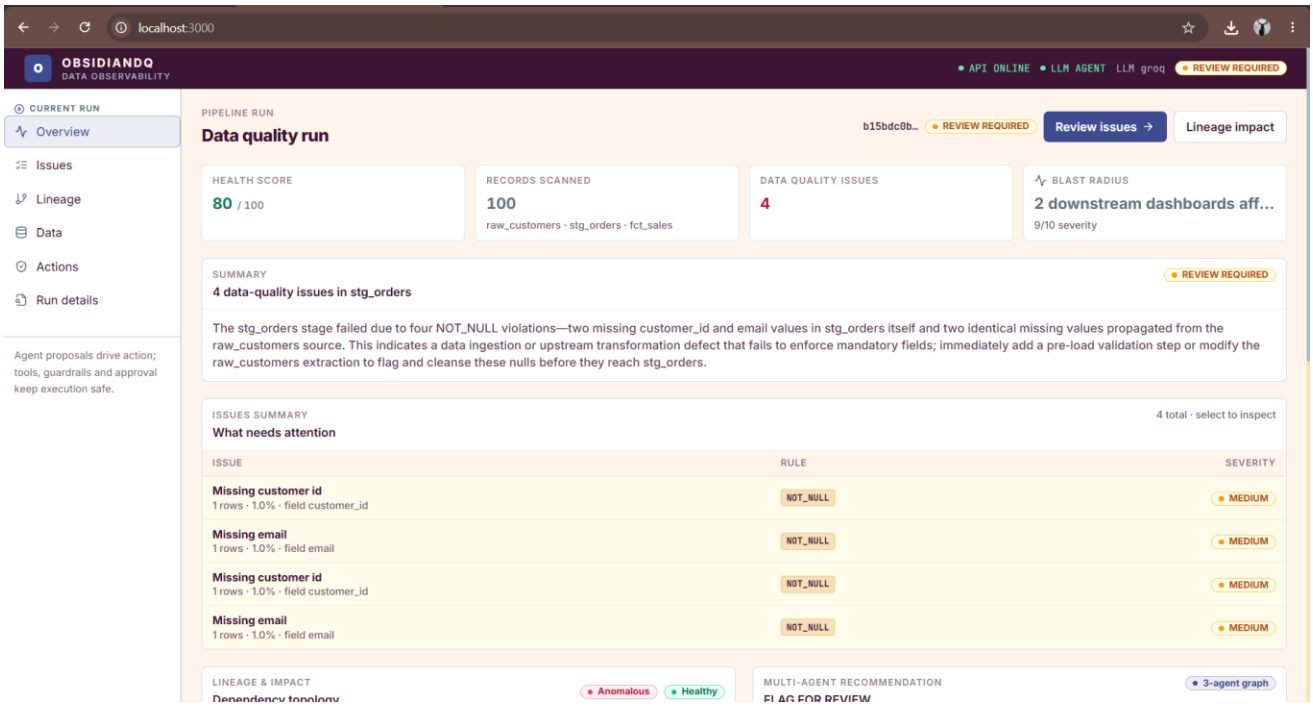


Figure 7.2: Screen 2 Executive Run Console Overview (RunConsole.tsx - Overview Tab)

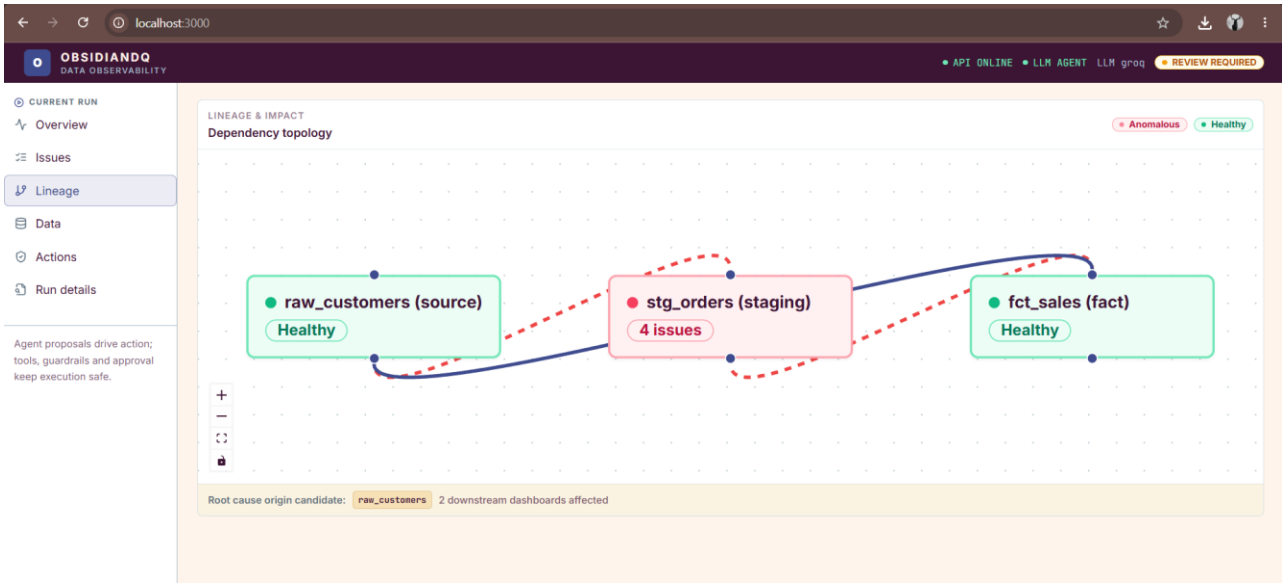
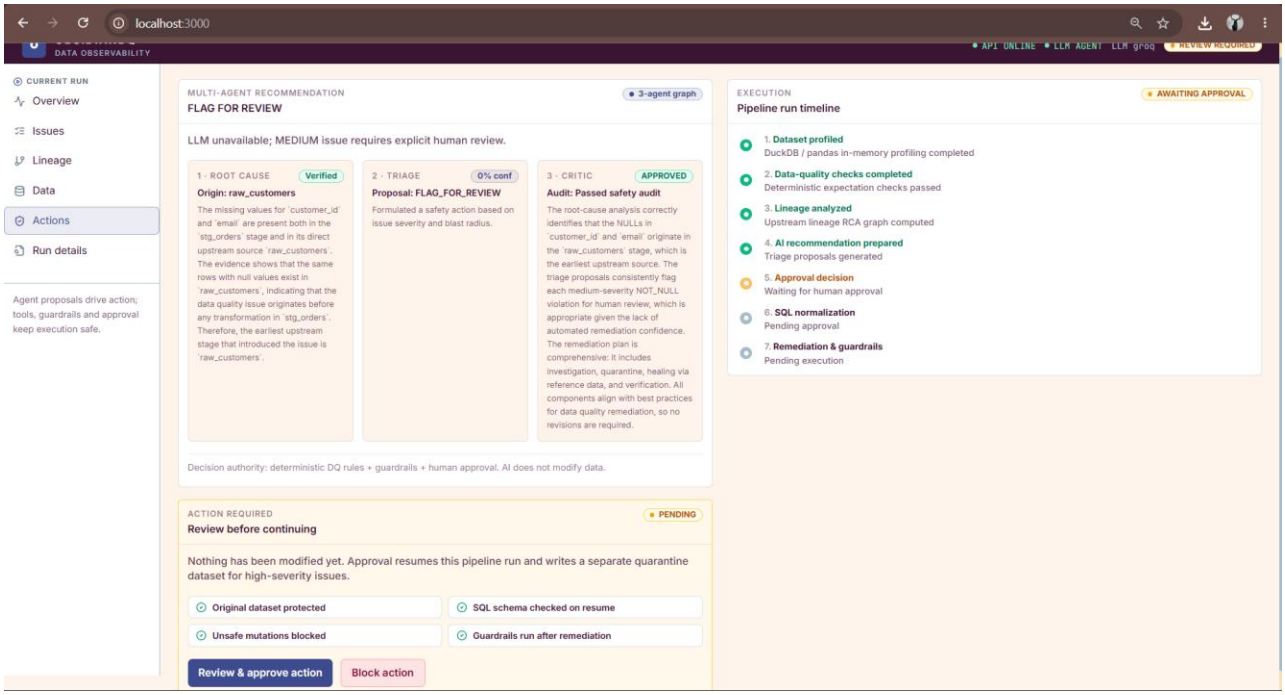


Figure 7.2: Screen 3 Interactive Lineage DAG View (LineageView.tsx)



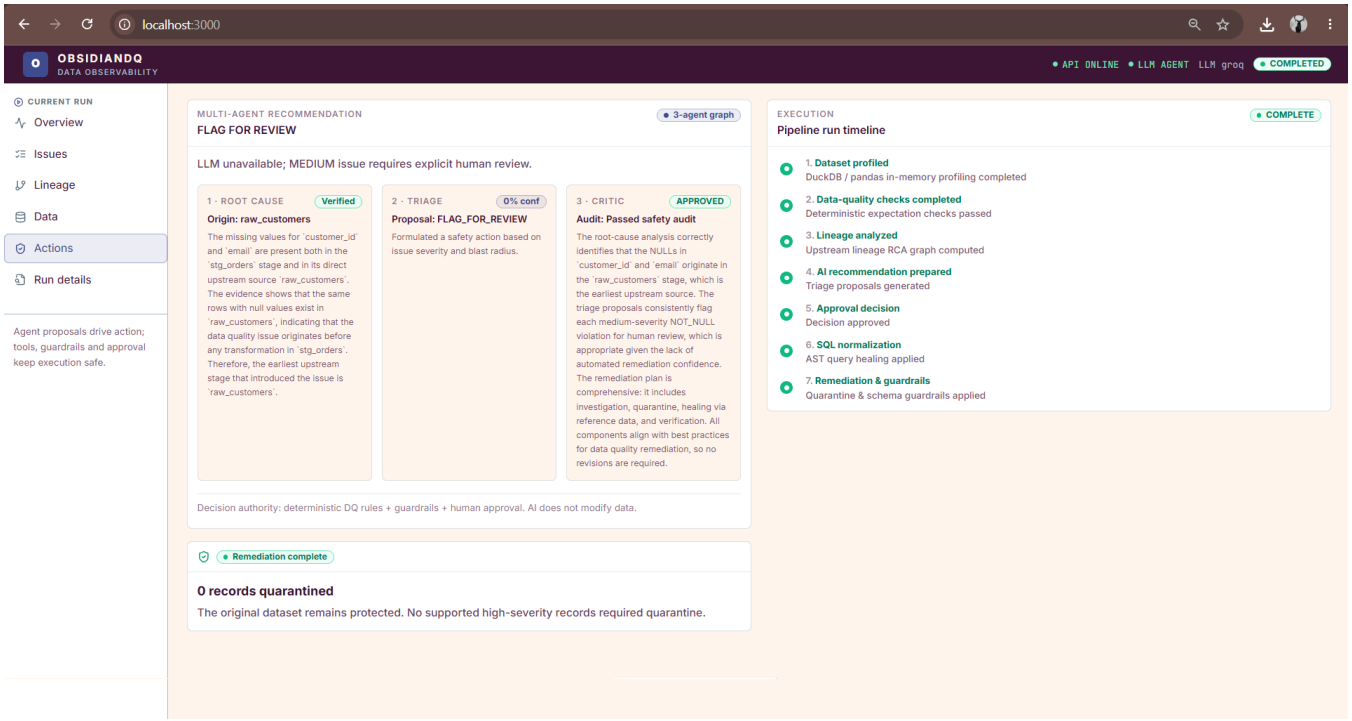


Figure 7.2: Screen 4 Multi-Agent Recommendation & Human Approval Gate (ActionViews.tsx)

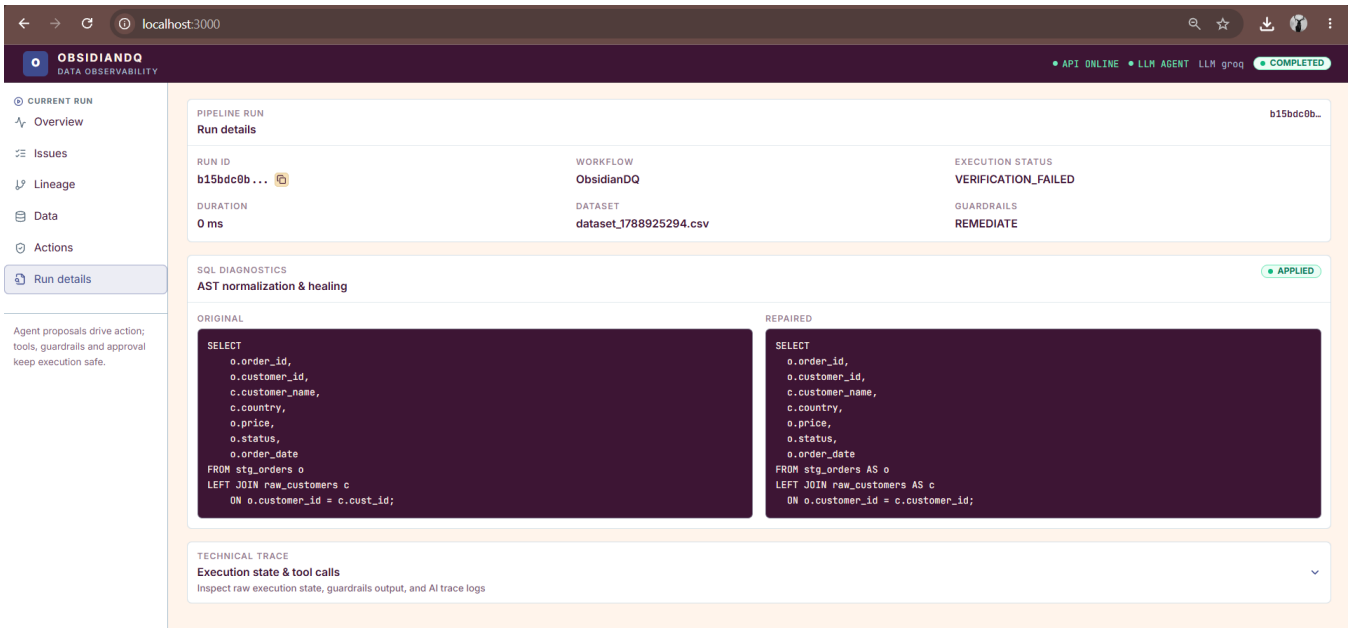


Figure 7.2: Screen 5 AST-Grounded SQL Healer Diff Viewer (RunConsole.tsx - Details Tab)

8. LIMITATIONS

While ObsidianDQ establishes a safe architecture for data quality investigation, several operational limitations remain:

1. **LLM Dependency and Rate Limiting:** Agentic reasoning depends on external API availability. Under free-tier Groq quotas, multi-turn tool calling frequently triggers HTTP 429 rate limits. While fail-closed fallbacks prevent pipeline crashes, they bypass live agent reasoning.
2. **Optimistic Critic Fallback:** In the current implementation, if the Critic Agent's LLM invocation fails or is unconfigured, it defaults to APPROVED rather than halting the workflow. While downstream guardrails and human gates catch high-risk plans, an ideal safety critic should fail closed.
3. **Absence of Automatic Rollback on Verification Failure:** If post-remediation verification detects remaining errors, it sets the status to VERIFICATION_FAILED and flags recovery requirements. However, it does not currently delete the invalid cleaned artifact or roll back quarantine files automatically.
4. **Volatile In-Memory Graph Checkpointing:** The current LangGraph runtime uses MemorySaver and stores run states in Python process memory (RUN_STATES dict). Terminating or restarting the FastAPI server clears paused runs, requiring persistent storage (e.g., Postgres or SQLite checkpointers).
5. **Schema-Specific Rule Coupling:** Core validation rules (PRICE_NON_NEGATIVE, ORDER_DATE_RANGE) are tailored to the e-commerce orders schema. Deploying the system across diverse business domains requires generic, configuration-driven rule definitions (e.g., via YAML or Great Expectations DSL).

9. CONCLUSION

ObsidianDQ demonstrates that modern AI-driven data engineering does not require choosing between brittle deterministic scripts and hazardous, unconstrained LLM writers. By architecting a **Hybrid Agentic Workflow**, ObsidianDQ establishes clear operational boundaries:

- Vectorized Python rule engines perform deterministic anomaly scanning.
- Lineage DAG traversal accurately isolates upstream corruption origins and downstream blast radius.
- Read-only multi-agent LLM teams inspect evidence, investigate distributions, formulate remediation plans, and audit safety.
- sqlglot AST parsing resolves broken SQL transformations without hallucinating syntax.
- Checkpointed Human-in-the-Loop gates ensure that high-risk actions are approved by human engineers.
- Dedicated quarantine partitions and post-remediation re-verification ensure zero raw data loss and truthful outcome reporting.

Across 33 automated contract tests and 20 controlled benchmark incidents, ObsidianDQ achieved **100.0% incident handling accuracy** and **100.0% verification consistency**, establishing a robust blueprint for governed, autonomous data observability.

10. FUTURE WORK

Subsequent development of ObsidianDQ will target four key technical extensions:

1. **Persistent Distributed Checkpointing:** Transition from in-memory MemorySaver to a distributed PostgreSQL or Redis checkpointer, enabling cross-node worker resumption and enterprise horizontal scaling.
2. **Dynamic DSL Rule Specification:** Replace hard-coded validation checks with a declarative YAML expectation engine compatible with Great Expectations and dbt-expectations.
3. **Automated Fault-Injection Test Harness:** Implement automated runtime fault injection (e.g., simulating disk write failures, database network partitions, and API timeouts) to systematically benchmark the **Failure Recovery Rate**.
4. **Semantic Root-Cause Ground Truth Benchmarks:** Curate a multi-stage enterprise dataset with human-labeled semantic root-cause explanations to quantitatively score LLM Root-Cause Precision, Recall, and F1.

REFERENCES

1. **LangChain & LangGraph Documentation:** *Building Stateful Multi-Agent Applications with Directed Graphs and Cyclic Control Flows.* LangChain AI, 2024.
2. **DuckDB Foundation:** *DuckDB: An In-Process Analytical SQL Database Management System.* Raasveldt, M., & Mühleisen, H., Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data, 2019.
3. **SQLGlot Project:** *An Exhaustive SQL Parser, Transpiler, Analyzer, and Engine.* Toby Mao et al., 2024. Available at: https://github.com/tobymao/sqlglot.
4. **FastAPI Documentation:** *Modern, Fast (High-Performance), Web Framework for Building APIs with Python.* Tiangolo, S., 2024.
5. **Next.js & React Framework:** *App Router Architecture, Server Components, and Streaming UI Rendering.* Vercel, 2024.
6. **Data Observability Principles:** *Data Quality Fundamentals: A Practitioner's Guide to Building Trustworthy Data Pipelines.* Moses, B., Gavish, L., & Vorwerck, D., O'Reilly Media, 2022.
7. **Automated Program Repair & AST Rewriting:** *A Survey on Automatic Software Repair.* Monperrus, M., ACM Computing Surveys (CSUR), 51(1), 1-37, 2018.
8. **AI Agents in Software Engineering:** *Communicative Agents for Software Development.* Qian, C. et al., arXiv preprint arXiv:2307.07924, 2023.
9. **Great Expectations Consortium:** *Standardizing Declarative Expectations for Automated Pipeline Testing.* Superconductive, 2023.
10. **Apache Airflow & Lineage Tracking:** *Data Lineage and Provenance in Directed Acyclic Orchestration Graphs.* Apache Software Foundation, 2024.